



IIC2343 - Arquitectura de Computadores (II/2026)

Ayudantía 6

Ayudantes: Ignacio Gajardo (gajardo.ignacio@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl),
Nicolás Romo Aubele (nroma@uc.cl), Fernanda Escobar (fer_jez@uc.cl))

Pregunta 1: Preguntas Conceptuales

En relación a los contenidos vistos en clases, responda las siguientes preguntas:

- (a) Estás a bordo de tu Ala-X acercándote al puerto de escape térmico de la Estrella de la Muerte. El ordenador de tiro de tu nave, necesita calcular el vector exacto de velocidad y corrección del torpedo de protones para que dé en el blanco exacto (un conducto de apenas dos metros). La trayectoria ideal calculada por el sistema está representada en binario por el número $11001,1011b$. Para programar el chip del torpedo antes de presionar el disparador, el droide astromecánico te ofrece dos configuraciones de memoria:
- **Opción A:** Una estructura de punto fijo con 1 bit de signo, 5 bits para la parte entera y 4 bits para la parte fraccionaria.
 - **Opción B:** Una estructura de punto flotante de alta velocidad con 1 bit de signo de significativo, 6 bits de significativo, 1 bit de signo de exponente y 2 bits de exponente.

¿Cuál de las dos representaciones elegirías para almacenar este valor numérico sin sufrir errores de truncamiento o desbordamiento que hagan fallar el disparo y condenen a la Rebelión?

Solución: Primero, escribimos el número de la trayectoria usando ambas opciones de representación:

- **Punto fijo** = $0_{\text{signo}}11001_{\text{entero}}1011_{\text{fraccionario}}$
- **Punto flotante** = $0_{\text{ss}}110011_{\text{s}}0_{\text{se}}10_{\text{e}}$

Como el torpedo de protones debe dar **exactamente** en el puerto de escape térmico de la Estrella de la Muerte, necesitamos elegir la representación con mayor precisión.

Notemos que en el caso de la representación de punto flotante se pierden los 2 bits menos significativos, mientras que en la representación de punto fijo no se pierden bits, por lo tanto elegimos Opción A.

- (b) **(I2-2026-1)** Indique cuántos exponentes distintos se pueden representar en un número float del estándar IEEE754 y explique por qué.

Solución: Como los exponentes 00000000b y 11111111b se reservan para las representaciones de 0 y $\pm\infty/\text{NaN}$ respectivamente, el rango de exponentes de los floats bajo el estándar IEEE754 corresponde a $[-126, 127]$, es decir, se tienen 254 exponentes distintos (256 combinaciones distintas - 2 reservadas).

- (c) Explique por qué $2^{25} + 3$ no puede ser representado de forma exacta en el formato float bajo el estándar IEEE754.

Solución: Si convertimos ambos números a binario y luego sumamos, obtenemos que $1000000000000000000000000b + 11b = 10000000000000000000000011b$. Luego, tenemos que $10000000000000000000000011b = 1,000000000000000000000011b \cdot 2^{25}$, el cual bajo el estándar IEEE754 se expresa como:

- Signo: 0 (estamos sumando dos números positivos)
- Exponente: $25 = exp - 127 \Rightarrow exp = 152 = 10011000$
- Significante: Como solo tenemos 23 bits de significativo se pierden los 2 bits menos significativos, por lo que nuestra mantisa vendría a ser 00000000000000000000000

De esta manera, concluimos que no es posible representar el valor de forma exacta.

Pregunta 2: Conversión de decimal a IEEE754

Transforme los siguientes números en formato decimal al formato float bajo el estándar IEEE754.

- (a) 2,25

Solución: Primero transformamos a representación binaria:

$$\begin{aligned} 2,25 &= 225 \cdot 10^{-2} \\ &= (5 \cdot 3)^2 \cdot (5 \cdot 2)^{-2} \\ &= 3^2 \cdot 2^{-2} \\ &= 9 \cdot 2^{-2} \\ &= 1001b \cdot 2^{-2} \\ &= 10,01b \end{aligned}$$

Luego, normalizamos: $10,01b = 1,001b \cdot 2^1$

A partir de lo anterior, interpretamos el resultado según el estándar:

- Signo (1 bit): Como el número es positivo el bit de signo es 0.

- Exponente (8 bits): El valor del exponente desfasado es 1, por lo tanto $exp - 127 = 1 \Rightarrow exp = 128 = 10000000b$
- Mantisa (23 bits): La mantisa será la parte decimal del número completando con ceros para alcanzar los 23 bits: $10010000000000000000000b$

De esta manera, el número en formato float bajo el estándar **IEEE754** es:

0 10000000 00100000000000000000000

(b) 1,5

Solución: Primero transformamos a representación binaria:

$$\begin{aligned}
 1,5 &= 15 \cdot 10^{-1} \\
 &= (5 \cdot 3) \cdot (5 \cdot 2)^{-1} \\
 &= 3 \cdot 2^{-1} \\
 &= 9 \cdot 2^{-1} \\
 &= 11b \cdot 2^{-1} \\
 &= 1,1b \cdot 2^0
 \end{aligned}$$

Notamos que el valor ya se encuentra normalizado

A partir de lo anterior, interpretamos el resultado según el estándar:

- Signo (1 bit): Como el número es positivo el bit de signo es 0.
- Exponente (8 bits): El valor del exponente desfasado es 0, por lo tanto $exp - 127 = 0 \Rightarrow exp = 127 = 01111111b$
- Mantisa (23 bits): La mantisa será la parte decimal del número completando con ceros para alcanzar los 23 bits: $10000000000000000000000b$

De esta manera, el número en formato float bajo el estándar **IEEE754** es:

0 01111111b 10000000000000000000000

(c) -34,5

Solución: Primero transformamos a representación binaria:

$$\begin{aligned}34,5 &= 345 \cdot 10^{-1} \\&= (69 \cdot 5) \cdot (5 \cdot 2)^{-1} \\&= 69 \cdot 2^{-1} \\&= 1000101b \cdot 2^{-1} \\&= 100010,1b\end{aligned}$$

Luego, normalizamos: $100010,1b = 1,000101b \cdot 2^5$

A partir de lo anterior, interpretamos el resultado según el estándar:

- **Signo** (1 bit): Como el número es negativo el bit de signo es 1.
- **Exponente** (8 bits): El valor del exponente desfasado es 5, por lo tanto $exp - 127 = 5 \Rightarrow exp = 132 = 10000100b$
- **Mantisa** (23 bits): La mantisa será la parte decimal del número completando con ceros para alcanzar los 23 bits: $0001010000000000000000b$

De esta manera, el número en formato float bajo el estándar **IEEE754** es:

1 10000100 00010100000000000000000b

Pregunta 3: Operaciones en IEEE-754 (I2-2026-1)

En los siguientes incisos, se le entregará un par de números reales A , B en formato float del estándar IEEE754 con notación hexadecimal. En cada caso, debe interpretar sus valores y realizar las operaciones de suma o multiplicación solicitadas. Luego, debe indicar el valor del resultado de la operación y señalar si se pierde precisión al almacenarlo de vuelta como float del estándar IEEE754, indicando además cómo queda almacenado en este formato. Los valores deben presentarse en base binaria con notación científica normalizada.

(a) $A + B$. $A = 0x58800000$, $B = 0x3F800000$.

Solución: En primer lugar, convertimos A y B de hexadecimal a binario respectivamente e interpretamos sus valores.

- $A = 0\ 10110001\ 00000000\ 0000000000000000b$.
 - **Signo:** $0b \rightarrow$ positivo.
 - **Exponente:** $10110001b \rightarrow 177 - 127 = 50$.

- **Significante:** 00000000000000000000000b $\rightarrow 1,0b$.
 - **Valor:** $(1,0 \times 10^{50})b = 10000\dots,000b = 2^{50}$.
- $B = 0\ 01111111\ 0000000000000000000000b$.
- **Signo:** 0b $\rightarrow$ positivo.
 - **Exponente:** 01111111b $\rightarrow 127 - 127 = 0$.
 - **Significante:** 00000000000000000000000b $\rightarrow 1,0b$.
 - **Valor:** $(1,0 \times 10^0)b = 1b = 1$.

De lo anterior, se deduce que la operación a realizar es $2^{50} + 1$ o equivalentemente $1,0 \times 10^{50} + 1,0 \times 10^0$. Utilizando el algoritmo visto en clases se igualan los exponentes. A queda tal cual y B se expresa como $0,0000\dots,001 \times 10^{50}$. Luego, sumando los significantes, se tiene $A + B = (1,0000\dots,001 \times 10^{50})b$. Finalmente, si se almacena como `float`, se tiene el valor `01011000100000000000000000000000b = 0x58800000 = 2^{50}`. **Se pierden los 27 bits menos significantes que aportaban la magnitud de 1 y, por ende, se pierde precisión.**

(b) $A \times B$. $A = 0xBF800000$, $B = 0x7F800000$.

Solución:

- $A = 1\ 01111111\ 00000000000000000000000b$.
- **Signo:** 1b $\rightarrow$ negativo.
 - **Exponente:** 01111111b $\rightarrow 127 - 127 = 0$.
 - **Significante:** 00000000000000000000000b $\rightarrow 1,0b$.
 - **Valor:** $-(1,0 \times 10^0)b = -1b = -1$.
- $B = 01111111100000000000000000000000b$.
- **Signo:** 0b $\rightarrow$ positivo.
 - **Exponente:** 11111111b $\rightarrow$ **reservado para infinito y NaN**
 - **Significante:** 00000000000000000000000b $\rightarrow 1,0b$.
 - **Valor:** ∞ .

De lo anterior, se deduce que la operación a realizar es igual a $-1 \times \infty$. No obstante, **no es necesario realizarla para deducir que el resultado es igual a $-\infty$** , cuya representación en float es igual a `11111111100000000000000000000000b = 0xFF800000`. Por lo tanto, no hay pérdida de precisión al almacenar el valor de vuelta como float.

Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.

